

	OVERVIEW	PRICING	DOCS ↗	EXAMPLES ↗	LOG IN	SIGN UP
Fine-tuned models						
Create your own custom models by fine-tuning our base models with your training data. Once you fine-tune a model, you'll be billed only for the tokens you use in requests to that model.						
LEARN MORE ↴						
	MODEL	TRAINING	USAGE			
	Ada	\$0.0004 / 1K tokens	\$0.0016 / 1K tokens			
	Babbage	\$0.0006 / 1K tokens	\$0.0024 / 1K tokens			
	Curie	\$0.0030 / 1K tokens	\$0.0120 / 1K tokens			
	Davinci	\$0.0300 / 1K tokens	\$0.1200 / 1K tokens			

Fig. 8: List of ChatGPT models

OpenAI and carry out an experiment with ChatGPT. Next, we import the modules for NLP. After that, we import *pygame* to play the file that processed the reply in a human voice using the NLP model.

Next, we need to set the ChatGPT model (see Fig. 8). Here, we can choose from model names like Davinci, Ada, etc. Each model has its own expertise, and the cost of using these models varies. But no worries, because developers get a US\$18 credit to develop and experiment with OpenAI.

Next, we need to set the API in the code. With that, we have created the function for connecting with ChatGPT to handle the query and get the response from it.

```
import speech_recognition as sr
import math
import time
import serial
from espeak import espeak
import sys
import openai
import pygame
from gtts import gTTS
pygame.mixer.init()
#model_to_use="text-davinci-003" # most
capable
#model_to_use="text-curie-001"
#model_to_use="text-babbage-001"
model_to_use="text-ada-001" # lowest
token cost
r = sr.Recognizer()
openai.api_key="*****Your Key
Here*****"
```

```
def chatGPT(query):
    response = openai.Completion.create(
        model=model_to_use,
        prompt=query,
        temperature=0,
        max_tokens 1000
    )
    return str.strip(response['choices'][0]
['text']), response['usage']['total_
tokens'])
```

After that, we create the main function and then make a *while* loop. Here, we use NLP to capture the voice continuously and extract what we said using the NLP model and save it as a query. Then we transfer this query to ChatGPT and receive the response from it.

```
def main():
    print('LED is ON while button is
pressed (Ctrl-C for exit).')
    while True:
        with sr.Microphone() as source:
            r.adjust_for_ambient_noise (source)
            print("Say something!")
            audio r.listen(source)
            print("Recognizing Now....")
            command=str(r.recognize_google (audio))
            print("Google Speech Recognition thinks
you said + command)
            query=command
            (res, usage) = chatGPT (query)
            print(res)
            tts gTTS(text=res, lang='en')
            tts.save("good.mp3")
            pygame.mixer.music.load("good.mp3")
            pygame.mixer.music.play()
            #espeak.synth(res)
```

```
if __name__ == '__main__':
    main()
```

After this, we again use NLP to convert the reply from ChatGPT into a human voice, and then we play that voice. This whole thing runs in a loop continuously making it look like a real conversation between two humans.

This VoiceGPT gives you the option of customising and selecting models. It allows you to choose from GPT models like Ada, Davinci, or Babbage. It uses a free speech-recognition service that can be customised to offline speech-recognition services like Sphenix.

Testing VoiceGPT

To test the VoiceGPT, run the code in Python, and it will tell you to ask a question or start a conversation. You can ask whatever you want; it recognises your voice, transfers the query to ChatGPT, and then replies to you in a human voice.

So now you can talk to ChatGPT just like you do with Google Assistant, Alexa, or Siri. Enjoy your conversation with VoiceGPT! **EFY**

Author's note. This is the first version of VoiceGPT. I am still experimenting with it and you will get all the new updates very soon on <https://www.opensourceforu.com>.

Ashwini Kumar Sinha is an IoT and AI enthusiast tech journalist at EFY